

OpsCore Secure Data Access Exercise

Graded Exercise — Student Packet · PostgreSQL

Learning Objectives

By completing this exercise, you will be able to:

- Identify SQL injection entry points (value slots vs. identifier slots) in a working PL/pgSQL function, and demonstrate a working exploit against each.
- Rewrite vulnerable dynamic SQL as parameterized functions, using bound values, array-based IN lists, and an allow-list for any identifier that cannot be bound.
- Prove that a fix actually works by re-running the original exploit and confirming it is now inert.

Setup

Run these two files, in order, in a scratch database (e.g. opscore_lab). Do not run them against any shared or production database.

- 00_opscore_schema.sql — the shared OpsCore schema (hosts, services, app_users, events, tickets). Provided by your instructor.
- opscore-exercise-starter.sql — adds a searchable summary column to tickets, seeds a few extra rows, and installs the two INSECURE functions you will audit in Part 1.

Do all of your Part 2 and Part 3 work inside the TODO blocks of opscore-exercise-starter.sql (or in a separate file you submit alongside it) — do not modify the two INSECURE functions themselves; you are auditing them as-is.

SUBMIT

(1) Your completed .sql file with all Part 2 and Part 3 code, and (2) this document with every "Your answer:" section filled in — pasted query output, exact payloads you used, and written responses.

Part 1 — Find the Injection

The two functions below are already installed by the starter file. Do not edit them — your job is to audit them, exploit them, and explain why each exploit works.

```
CREATE OR REPLACE FUNCTION usp_lookup_tickets_by_status_INSECURE(p_status TEXT)
RETURNS SETOF tickets LANGUAGE plpgsql AS $$
DECLARE v_sql TEXT;
BEGIN
    v_sql := 'SELECT * FROM tickets WHERE status = ''' || p_status || ''';
    RAISE NOTICE 'Assembled statement: %', v_sql;
    RETURN QUERY EXECUTE v_sql;
END; $$;
```

```
CREATE OR REPLACE FUNCTION usp_search_tickets_INSECURE(p_keyword TEXT, p_sort_col
TEXT)
RETURNS TABLE(ticket_id INT, kind TEXT, status TEXT, priority TEXT, summary TEXT,
opened_at TEXT)
LANGUAGE plpgsql AS $$
DECLARE v_sql TEXT;
BEGIN
    v_sql := 'SELECT ticket_id, kind::text, status::text, priority::text,
summary::text, opened_at::text '
        || 'FROM tickets WHERE summary ILIKE ''' || p_keyword || ''''
        || 'ORDER BY ' || p_sort_col;
    RAISE NOTICE 'Assembled statement: %', v_sql;
    RETURN QUERY EXECUTE v_sql;
END; $$;
```

1.1 Classify the entry points

For each snippet below, write whether the highlighted input lands in a VALUE slot or an IDENTIFIER slot, and whether the snippet as shown is vulnerable or already safe. One sentence of justification each.

- (a) ...WHERE priority = '' || p_priority || ''
- (b) ...ORDER BY ' || p_column
- (c) ...WHERE priority = p_priority (p_priority is a function parameter, no concatenation)
- (d) ...WHERE summary ILIKE '%' || p_keyword || '%'

Your answer:

(a) VALUE slot — vulnerable. The parameter is concatenated directly into a quoted SQL string, allowing SQL syntax to be injected.

(b) IDENTIFIER slot — vulnerable. The parameter is concatenated into an ORDER BY clause, where it becomes SQL syntax rather than a data value.

(c) VALUE slot — safe. The parameter is used as a PL/pgSQL value instead of being concatenated into SQL text.

(d) VALUE slot — safe as written. The keyword is treated as a value expression rather than SQL syntax; in dynamic SQL, it should still be supplied through a bound parameter.

1.2 Exploit the tautology

Craft a single call to `usp_lookup_tickets_by_status_INSECURE(...)` that returns every ticket in the table regardless of its actual status. Run it, then paste (a) the assembled statement from the RAISE NOTICE line, (b) the number of rows returned, and (c) a 1–2 sentence explanation of why the WHERE clause no longer filters anything.

Your answer: **(a) Assembled statement:**

SELECT * FROM tickets WHERE status = " OR '1'='1'

(b) Number of rows returned: 7

(c) Explanation:

The payload closes the original quoted status value and adds the condition '1'='1', which is always true. Therefore, the WHERE clause no longer restricts the result to a particular ticket status, so every ticket (7) is returned.

1.3 Exploit the UNION-based read

Craft a call to `usp_search_tickets_INSECURE(...)` whose `p_keyword` value closes the ILIKE pattern and appends a UNION SELECT that pulls username and email out of `app_users`. Paste the assembled statement and the result rows, and state exactly which two real columns were exfiltrated and which positions the NULL placeholders had to fill.

Your answer:

Payload used for p_keyword:

%' UNION SELECT NULL, username, email, NULL, NULL, NULL FROM app_users --

Assembled statement:

**SELECT ticket_id, kind::text, status::text, priority::text, summary::text, opened_at::text
FROM tickets**

WHERE summary ILIKE '%%'

UNION SELECT NULL, username, email, NULL, NULL, NULL FROM app_users

--%' ORDER BY ticket_id

Result rows from the UNION injection:

NULL "jordan" "jordan@example.com" NULL NULL NULL
NULL "hae" "hae@example.com" NULL NULL NULL
NULL "amal" "amal@example.com" NULL NULL NULL

The normal ticket rows were also returned by the original query.

Columns exfiltrated:

- app_users.username — placed in position 2, which corresponds to the original kind column.
- app_users.email — placed in position 3, which corresponds to the original status column.

NULL placeholder positions:

The injected UNION SELECT had to return the same six-column shape as the original query. Therefore, NULL filled positions 1, 4, 5, and 6, corresponding to ticket_id, priority, summary, and opened_at.

The exploit worked because p_keyword was concatenated directly into the dynamic SQL. The payload terminated the original ILIKE string, added a UNION SELECT against app_users, and used -- to comment out the remainder of the original SQL statement.

Part 2 — Parameterize

Write each of the following in your .sql file. None of them should build a query string by concatenating a value into it.

2.1 `usp_lookup_tickets_by_status(p_status TEXT)`

A secured replacement for artifact A. This one needs no dynamic SQL at all — a plain parameterized query is enough.

2.2 `usp_search_tickets(p_keyword TEXT, p_sort_key TEXT)`

- Bind the keyword as a value — never concatenate it into the statement text.
- `p_sort_key` is structure, not a value: allow-list it to one of 'opened_at', 'priority', 'status' with a CASE expression; anything else should default to 'opened_at' rather than erroring.
- Use `format()` with the `%I` specifier to compose the validated identifier into the ORDER BY clause.

2.3 `tickets_by_priority(p_priorities TEXT[])`

Accept an array of priority codes (e.g. `ARRAY['p1','p2']`) and return every ticket whose priority is in that array, using `= ANY(...)`.

2.4 Prove the fix

Re-run your exact payloads from 1.2 and 1.3 against the two new secured functions. Paste both results and state, in one sentence each, why each payload is now inert.

Your answer:

2.1

```
CREATE OR REPLACE FUNCTION usp_lookup_tickets_by_status(p_status TEXT)
RETURNS SETOF tickets
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT *
    FROM tickets
    WHERE status = p_status;
END;
$$;
```

2.2

```
CREATE OR REPLACE FUNCTION usp_search_tickets(
    p_keyword TEXT,
    p_sort_key TEXT
```

```

)
RETURNS TABLE(
    ticket_id INT,
    kind TEXT,
    status TEXT,
    priority TEXT,
    summary TEXT,
    opened_at TEXT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_sort_col TEXT;
    v_sql TEXT;
BEGIN
    v_sort_col :=
        CASE p_sort_key
            WHEN 'opened_at' THEN 'opened_at'
            WHEN 'priority' THEN 'priority'
            WHEN 'status' THEN 'status'
            ELSE 'opened_at'
        END;

    v_sql := format(
        'SELECT ticket_id, kind::text, status::text, priority::text,
          summary::text, opened_at::text
        FROM tickets
        WHERE summary ILIKE $1
        ORDER BY %!',
        v_sort_col
    );

    RETURN QUERY EXECUTE v_sql
        USING '%' || p_keyword || '%';
END;
$$;

```

2.3

```

CREATE OR REPLACE FUNCTION tickets_by_priority(p_priorities TEXT[])
RETURNS SETOF tickets
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT *
    FROM tickets
    WHERE priority = ANY(p_priorities);
END;
$$;

```

2.4

Test 1 — Tautology payload

```

SELECT *
FROM usp_lookup_tickets_by_status('' OR "1"="1");

```

Result: 0 rows returned.

Why the payload is inert: The payload is treated as a literal TEXT value rather than SQL syntax because p_status is used directly as a parameter in the query.

Test 2 — UNION payload

SELECT *
FROM usp_search_tickets(
"%" UNION SELECT NULL, username, email, NULL, NULL, NULL FROM app_users --",
'ticket_id'
);

Result: 0 rows returned; no usernames or email addresses from app_users were returned.

Why the payload is inert: The keyword is passed as a bound \$1 parameter, so the injected UNION SELECT is treated as literal search text rather than executable SQL.

Part 3 — Reflection

Answer each in 2–4 sentences, in your own words.

3.1 Why can a bound parameter never become code?

Explain why a value bound through a parameter — even one containing quotes or SQL keywords — can never change the structure of the query it's bound into.

Your answer:

A bound parameter is sent to the database separately from the SQL statement, so PostgreSQL treats it only as data. Even if the value contains quotes, SQL keywords, or something like OR '1'='1', it cannot change the already-defined structure of the query. The database does not re-parse the parameter value as part of the SQL code.

3.2 Why isn't the sort column "just another parameter"?

In `usp_search_tickets`, the keyword is bound directly, but the sort column goes through an allow-list instead of a parameter. Explain what would go wrong if you tried to bind `p_sort_key` directly as a parameter to `ORDER BY`, and why concatenating it raw (without the allow-list) would be dangerous.

Your answer:

The sort column represents part of the query's structure, because ORDER BY needs a column identifier rather than a data value. If p_sort_key were bound as a normal parameter, PostgreSQL would treat it as a literal value instead of interpreting it as the name of the column to sort by. Concatenating it directly without an allow-list would be dangerous because an attacker could supply SQL syntax instead of a valid column name, potentially changing the query.

3.3 What would undo your Part 3 protections?

Suppose `helpdesk_app` were accidentally also granted PostgreSQL's built-in `pg_read_all_data` role. What would that undo, and why does the view you built stop being an effective boundary in that scenario?

Your answer:

Granting helpdesk_app the pg_read_all_data role would give it broad read access to all tables and views in the database, undoing the least-privilege protection we established. The vw_helpdesk_tickets view would no longer be an effective security boundary because the role could bypass the view and directly query tables that the view was intended to restrict. In other words, the view can limit what the application is allowed to access only if the database role does not already have broader privileges.

3.4 Why doesn't "escape the quotes" work as a defense?

Give one concrete situation (from this exercise or the lessons) where escaping quotes or blocking specific keywords would fail to stop an injection that parameterization stops.

Your answer:

Escaping quotes or blocking specific keywords is not a reliable defense because attackers can often find another way to change the SQL syntax or bypass the blocked pattern. In this exercise, an attacker could inject SQL through the p_sort_col value in the ORDER BY clause without needing the same quote-based payload used for the status search. Parameterization prevents this because values are kept separate from the SQL statement, so the input cannot become executable SQL regardless of its contents.
